

Lab 4 - Vibe Coding chat history (PES2UG24AM154)

Scenario 04 - Target Shooting · Claude Code session (Claude Opus 5.5) · excerpt of the session export dated 2026-10-01

Excerpt notice: This file is an excerpt of the exported session titled 'Lab 4 - Vibe Coding chat history (PES2UG24AM093)'. That session covered several students' repositories in one conversation. Only the parts that concern PES2UG24AM154 (04_target_shooting) are kept below; text is unchanged from the export. Removed passages are marked [... omitted: other SRNs ...]. Tool outputs longer than 30 lines were already truncated in the original export.

USER

take thwse projects they need to be vibe coded do it for these partiucular srns PES2UG24AM093:
https://github.com/SETAPESU26/14_2048 AM154: https://github.com/SETAPESU26/04_target_shooting AND
AM091: https://github.com/SETAPESU26/12_maze_runner

CLAUDE

I'll start by looking at the three repos to see what each assignment asks for.

> Bash Clone the three assignment repos and list contents

```
mkdir -p ~/setapesu26 && cd ~/setapesu26 && for r in 14_2048 04_target_shooting 12_maze_runner; do gh repo clone SETAPESU26/$r 2>&1 | tail -2; done; ls -la */ ; ...
```

```
Cloning into '04_target_shooting'...
04_target_shooting/:
total 24
drwxr-xr-x  6 user staff   192 1 Oct 11:50 .
drwxr-xr-x  5 user staff   160 1 Oct 11:50 ..
drwxr-xr-x 12 user staff   384 1 Oct 11:50 .git
-rw-r--r--  1 user staff    31 1 Oct 11:50 .gitignore
-rw-r--r--  1 user staff  4099 1 Oct 11:50 README.md
drwxr-xr-x  5 user staff   160 1 Oct 11:50 target-shooting
```

[... omitted: output for other SRNs' repositories ...]

> Bash Print all READMEs and source files

[... omitted: README and source listings for the other SRNs' repositories ...]

CLAUDE

All three are lab assignments that explicitly allow LLM use, so I'll do them. Before I start, I'll check that Python and pygame are available.

> Bash Check Python and pygame availability

```
cd ~/setapesu26 && python3 --version; python3 -c "import pygame; print(pygame.version.ver)" 2>&1 | tail -1; which
python3.12
python3.11

Exit code 1
Python 3.14.7
ModuleNotFoundError: No module named 'pygame'
/opt/homebrew/bin/python3.12
python3.11 not found
```

> Bash Create venv and install pygame

```
cd ~/setapesu26 && python3.12 -m venv .venv && .venv/bin/pip install -q "pygame>=2.5" 2>&1 | tail -2; .venv/bin/python
-c "import
pygame; print(pygame.version.ver)"

[notice] A new release of pip is available: 25.1.1 -> 26.2.1
[notice] To update, run: /Users/user/setapesu26/.venv/bin/python3.12 -m pip install --upgrade pip
pygame 2.6.1 (SDL 2.28.4, Python 3.12.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
2.6.1
```

[... omitted: 2048 bug reproduction and fix (other SRN) ...]

CLAUDE

2048 is done. Next, Target Shooting. Before changing anything, I'll confirm the hitbox bug numerically:

> Bash Reproduce the target hitbox offset bug

```
cd ~/setapesu26/04_target_shooting/target-shooting && SDL_VIDEODRIVER=dummy ../../.venv/bin/python - <<'EOF' 2>&1 |
grep -v pygame
from game.target import Target
from game.hit_detection import check_hit
t = Target(200, 200, radius=28)
```

```

for label, p in [("center", (200,200)), ("inside left edge", (175,200)), ("inside top edge", (200,175)),
                ("outside right (x+40)", (240,200)), ("outside bottom-right corner", (250,250))]:
    print(f"{label:30s} {p} -> {'HIT' if check_hit([t], p) else 'miss'}")
EOF

center                (200, 200) -> HIT
inside left edge      (175, 200) -> miss
inside top edge       (200, 175) -> miss
outside right (x+40)  (240, 200) -> HIT
outside bottom-right corner (250, 250) -> HIT

```

CLAUDE

Confirmed: clicks inside the left/top edges miss, and clicks outside to the bottom-right register as hits. Writing the fix and the three features:

> Write /Users/user/setapesu26/04_target_shooting/target-shooting/game/hit_detection.py

```

"""
hit_detection: figures out whether a click landed on a target.
"""

def point_in_circle(px, py, cx, cy, radius):
    """True if (px, py) lies inside or on the circle centred at (cx, cy)."""
    dx = px - cx
    dy = py - cy
    return dx * dx + dy * dy <= radius * radius

def check_hit(targets, click_pos):
    """
    Returns the target that was clicked, or None if the click missed
    every target. Uses the target's real circular shape (x, y is the
    centre), not a square bounding box. Targets drawn later sit on top,
    so they are checked first when targets overlap.
    """
    px, py = click_pos
    for target in reversed(targets):
        if point_in_circle(px, py, target.x, target.y, target.radius):
            return target
    return None

```

The file /Users/user/setapesu26/04_target_shooting/target-shooting/game/hit_detection.py has been updated successfully. (file state is current in your context – no need to Read it back)

> Write /Users/user/setapesu26/04_target_shooting/target-shooting/game/target.py

```

"""
Target: a circular target the player clicks on. (x, y) is the CENTER
of the circle - this matters for how it's drawn vs. how it's hit-tested.

Targets move inside a play-area rect and bounce off its edges. Two
movement patterns are supported:
- "linear": straight line at a constant speed (slow or fast)
- "wave": drifts horizontally while bobbing up and down on a sine wave
"""

import math
import pygame

PATTERN_COLORS = {
    "slow": (230, 90, 70),
    "fast": (240, 180, 50),
    "wave": (90, 170, 240),
}

class Target:
    def __init__(self, x, y, radius=28, color=(230, 90, 70),
                 vx=0.0, vy=0.0, pattern="linear", wave_amp=0.0, wave_freq=0.0):
        self.x = x
        self.y = y
        self.radius = radius
        self.color = color
        self.vx = vx # pixels per second
        self.vy = vy
        self.pattern = pattern
        self.wave_amp = wave_amp # pixels per second of vertical bob
        self.wave_freq = wave_freq # radians per second
        self.age = 0.0
    def get_bounding_rect(self):

```

```

        """Square bounding box around the circle, built from the centre."""
        return pygame.Rect(int(self.x - self.radius), int(self.y - self.radius),
                            self.radius * 2, self.radius * 2)

def update(self, dt, bounds):
    """Advance dt seconds and bounce off the edges of `bounds` (a Rect)."""
    self.age += dt
    dy = self.vy
    if self.pattern == "wave":
        dy += self.wave_amp * math.cos(self.wave_freq * self.age)
    self.x += self.vx * dt
    self.y += dy * dt

    left = bounds.left + self.radius
    right = bounds.right - self.radius
    top = bounds.top + self.radius
    bottom = bounds.bottom - self.radius
    if self.x < left:
        self.x = left + (left - self.x)
        self.vx = abs(self.vx)
    elif self.x > right:
        self.x = right - (self.x - right)
        self.vx = -abs(self.vx)
    if self.y < top:
        self.y = top + (top - self.y)
        self.vy = abs(self.vy)
    elif self.y > bottom:
        self.y = bottom - (self.y - bottom)
        self.vy = -abs(self.vy)
... (15 more lines)

```

The file /Users/user/setapesu26/04_target_shooting/target-shooting/game/target.py has been updated successfully. (file state is current in your context – no need to Read it back)

CLAUDE

Now the engine with scoring, combo and the timed round:

> Write /Users/user/setapesu26/04_target_shooting/target-shooting/game/game_engine.py

```

"""
GameEngine: owns the targets, the score/combo and the round timer, and
handles player input.

- Targets move (slow line, fast line, or sine-wave) and bounce off the
  edges of the play area.
- Each hit scores BASE_POINTS * combo multiplier; consecutive hits raise
  the multiplier (up to MAX_MULTIPLIER), a miss resets it to x1.
- A round lasts ROUND_SECONDS. When it ends no more shots are accepted
  and the final score is shown; R / SPACE starts a new round.
"""

import math
import random

import pygame

from game.target import Target, PATTERN_COLORS
from game.hit_detection import check_hit
from game.renderer import WIDTH, HEIGHT, HUD_HEIGHT

NUM_TARGETS = 3
TARGET_RADIUS = 28
ROUND_SECONDS = 30
BASE_POINTS = 10
MAX_MULTIPLIER = 5
HITS_PER_LEVEL = 3    # consecutive hits needed to raise the multiplier by 1

PLAY_AREA = pygame.Rect(0, HUD_HEIGHT, WIDTH, HEIGHT - HUD_HEIGHT)

# name: (min speed, max speed) in pixels per second
SPEEDS = {"slow": (70, 110), "fast": (190, 250), "wave": (90, 130)}

class GameEngine:
    def __init__(self, clock=pygame.time.get_ticks):
        self.clock = clock # milliseconds; injectable for testing
        self.start_round()

    # ---- round lifecycle -----

    def start_round(self):
        self.targets = [self._random_target() for _ in range(NUM_TARGETS)]
        self.hits = 0
        self.misses = 0
        self.score = 0
        self.streak = 0
        self.best_streak = 0
        self.round_over = False

```

```

        self.round_start_ms = self._clock()

    def time_left(self):
        elapsed = (self._clock() - self.round_start_ms) / 1000
        return max(0.0, ROUND_SECONDS - elapsed)

# ---- scoring -----

@property
def multiplier(self):
    return min(1 + self.streak // HITS_PER_LEVEL, MAX_MULTIPLIER)
... (69 more lines)

```

The file /Users/user/setapesu26/04_target_shooting/target-shooting/game/game_engine.py has been updated successfully.
(file state is current in your context – no need to Read it back)

> Write /Users/user/setapesu26/04_target_shooting/target-shooting/game/renderer.py

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)
HUD_HEIGHT = 40

COLOR_BG = (25, 25, 35)
COLOR_HUD = (40, 40, 58)
COLOR_TEXT = (255, 255, 255)
COLOR_ACCENT = (255, 220, 80)
COLOR_WARN = (255, 90, 90)

def draw_scene(surface, targets):
    surface.fill(COLOR_BG)
    for target in targets:
        center = (int(target.x), int(target.y))
        pygame.draw.circle(surface, target.color, center, target.radius)
        pygame.draw.circle(surface, (255, 255, 255), center, target.radius * 2 // 3, 2)
        pygame.draw.circle(surface, (255, 255, 255), center, target.radius // 3)
        pygame.draw.circle(surface, (255, 255, 255), center, target.radius, 2)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_hud(surface, font, score, multiplier, streak, hits, misses, time_left):
    pygame.draw.rect(surface, COLOR_HUD, (0, 0, surface.get_width(), HUD_HEIGHT))
    draw_text(surface, font, f"Score: {score}", (10, 9))
    combo_color = COLOR_ACCENT if multiplier > 1 else COLOR_TEXT
    draw_text(surface, font, f"Combo: x{multiplier} ({streak})", (170, 9), combo_color)
    draw_text(surface, font, f"Hits: {hits} Misses: {misses}", (380, 9))
    time_color = COLOR_WARN if time_left <= 5 else COLOR_TEXT
    secs = font.render(f"{time_left:4.1f}s", True, time_color)
    surface.blit(secs, (surface.get_width() - secs.get_width() - 10, 9))

def draw_round_over(surface, big_font, font, score, hits, misses, best_streak):
    overlay = pygame.Surface(surface.get_size(), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 170))
    surface.blit(overlay, (0, 0))
    shots = hits + misses
    accuracy = 100 * hits / shots if shots else 0
    cx, cy = surface.get_width() // 2, surface.get_height() // 2
    lines = [
        (big_font, "TIME'S UP!", COLOR_ACCENT, -80),
        (big_font, f"Final score: {score}", COLOR_TEXT, -30),
        (font, f"Hits {hits} Misses {misses} Accuracy {accuracy:.0f}%", COLOR_TEXT, 20),
        (font, f"Best streak: {best_streak}", COLOR_TEXT, 50),
        (font, "Press R or SPACE for a new round", COLOR_ACCENT, 100),
    ]
    for f, text, color, dy in lines:
        surf = f.render(text, True, color)
        surface.blit(surf, surf.get_rect(center=(cx, cy + dy)))
... (5 more lines)

```

The file /Users/user/setapesu26/04_target_shooting/target-shooting/game/renderer.py has been updated successfully.
(file state is current in your context – no need to Read it back)

> Write /Users/user/setapesu26/04_target_shooting/target-shooting/main.py

```
"""
Target Shooting

Run with:  python3 main.py

Click on targets to hit them. Consecutive hits build a combo; a miss
resets it. Each round lasts 30 seconds - press R or SPACE to play again.
"""
```

```
import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Target Shooting")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)
    big_font = pygame.font.SysFont("consolas", 40, bold=True)

    engine = GameEngine()
    running = True
    while running:
        dt = clock.tick(60) / 1000
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                engine.handle_click(event.pos)
            elif event.type == pygame.KEYDOWN:
                engine.handle_key(event.key)

        engine.update(dt)
        engine.draw(screen, font, big_font)

        pygame.display.flip()

    pygame.quit()

if __name__ == "__main__":
    main()
```

The file /Users/user/setapesu26/04_target_shooting/target-shooting/main.py has been updated successfully. (file state is current in your context – no need to Read it back)

CLAUDE

Now tests for target shooting, run headless with SDL's dummy video driver:

> Write /Users/user/setapesu26/04_target_shooting/target-shooting/test_game.py

```
"""Headless tests for the Target Shooting lab. Run with: python -m unittest -v"""
import os
import unittest

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
import pygame # noqa: E402

from game.game_engine import GameEngine, PLAY_AREA, ROUND_SECONDS, BASE_POINTS # noqa: E402
from game.hit_detection import check_hit # noqa: E402
from game.target import Target # noqa: E402

class FakeClock:
    def __init__(self):
        self.ms = 0

    def __call__(self):
        return self.ms

class HitDetectionTests(unittest.TestCase):
    def setUp(self):
        self.t = Target(200, 200, radius=28)

    def test_inside_edges_hit(self):
```

```

        for p in [(200, 200), (173, 200), (227, 200), (200, 173), (200, 227), (228, 200)]:
            self.assertIs(check_hit([self.t], p), self.t, p)

def test_outside_misses(self):
    # (240,200) and (250,250) used to register as hits with the shifted box.
    for p in [(229, 200), (240, 200), (250, 250), (171, 200), (221, 221)]:
        self.assertIsNone(check_hit([self.t], p), p)

def test_square_corner_is_a_miss(self):
    # Inside the bounding square but outside the circle.
    self.assertIsNone(check_hit([self.t], (226, 226)))

def test_bounding_rect_centered(self):
    self.assertEqual(self.t.get_bounding_rect().center, (200, 200))

class MovementTests(unittest.TestCase):
    def test_targets_stay_in_play_area(self):
        eng = GameEngine(clock=FakeClock())
        for _ in range(5000):
            eng.update(1 / 60)
            for t in eng.targets:
                self.assertTrue(PLAY_AREA.left + t.radius <= t.x <= PLAY_AREA.right - t.radius)
                self.assertTrue(PLAY_AREA.top + t.radius <= t.y <= PLAY_AREA.bottom - t.radius)

    def test_targets_actually_move_and_patterns_vary(self):
        eng = GameEngine(clock=FakeClock())
        before = [(t.x, t.y) for t in eng.targets]
        eng.update(0.05)
        self.assertNotEqual(before, [(t.x, t.y) for t in eng.targets])
        kinds = {eng._random_target().color for _ in range(200)}
        self.assertGreaterEqual(len(kinds), 2)

    def test_moving_target_can_be_hit(self):
        eng = GameEngine(clock=FakeClock())
... (83 more lines)

```

File created successfully at: /Users/user/setapesu26/04_target_shooting/target-shooting/test_game.py (file state is current in your context – no need to Read it back)

> Bash Run target shooting tests

```
cd ~/setapesu26/04_target_shooting/target-shooting && ../../.venv/bin/python -m unittest 2>&1 | tail -15
```

```

.....
-----
Ran 13 tests in 0.278s

OK
pygame 2.6.1 (SDL 2.28.4, Python 3.12.11)
Hello from the pygame community. https://www.pygame.org/contribute.html

```

[... omitted: maze test runs for other SRNs ...]

CLAUDE

Next I'll render headless screenshots of each game so I can check the visuals myself.

> Bash Render headless screenshots of each game state

```

[... screenshot script: Target Shooting part shown in the images below; Maze Runner part omitted ...]
['maze_fog_hint.png', 'maze_hint_nofog.png', 'maze_menu.png', 'maze_win.png', 'target_playing.png',
 'target_round_over.png']

```

> Read /Users/user/setapesu26/screenshots/target_playing.png



> Read /Users/user/setapesu26/screenshots/target_round_over.png



[... omitted: maze screenshots for other SRNs ...]

> Bash Apply visual fixes and rerun both test suites

```
[... Maze Runner edits omitted ...]
p = "04_target_shooting/target-shooting/game/renderer.py"
s = open(p).read()
old = s[s.index("def draw_hud"):s.index("def draw_round_over")]
new = '''def draw_hud(surface, font, score, multiplier, streak, hits, misses, time_left):
pygame.draw.rect(surface, COLOR_HUD, (0, 0, surface.get_width(), HUD_HEIGHT))
time_color = COLOR_WARN if time_left <= 5 else COLOR_TEXT
secs = font.render(f"{time_left:4.1f}s", True, time_color)
surface.blit(secs, (surface.get_width() - secs.get_width() - 12, 9))
combo_color = COLOR_ACCENT if multiplier > 1 else COLOR_TEXT
items = [
    (f"Score: {score}", COLOR_TEXT),
    (f"Combo x{multiplier} ({streak})", combo_color),
    (f"Hits {hits} Miss {misses}", COLOR_TEXT),
]
x = 10
for text, color in items:
    surf = font.render(text, True, color)
    surface.blit(surf, (x, 9))
    x += surf.get_width() + 26
```

```
'''
s = s.replace(old, new)
open(p, "w").write(s)

[... Maze Runner test run omitted ...]
cd ../04_target_shooting/target-shooting && ../../.venv/bin/python -m unittest 2>&1 | grep -E "^(OK|FAIL)"

OK
```

[... omitted: Maze Runner / 2048 / Maze Chase work for other SRNs ...]

CLAUDE

I've finished all three projects. Each one has the starter bug fixed and all four tasks built. Every test suite passes (13 tests for Target Shooting), and I checked the visuals with headless screenshots in ~/setapesu26/screenshots/.

AM154 - Target Shooting (04_target_shooting/target-shooting) - Bug: the hitbox was shifted down and to the right. Clicks inside a target's left or top edge missed, and clicks outside its bottom-right registered as hits. It now uses a proper circle test. - Tasks: - Targets move in three ways (slow, fast, and a wave pattern) and bounce off the edges. - Combo scoring: the multiplier goes up every 3 consecutive hits, up to x5, and a miss resets it. - A 30-second round, then a final-score screen; R or SPACE starts a new round.

To run: pygame is installed in ~/setapesu26/.venv . For example: cd
~/setapesu26/04_target_shooting/target-shooting && ../../.venv/bin/python main.py .

[... omitted: summaries for other SRNs, and the later Maze Chase request ...]